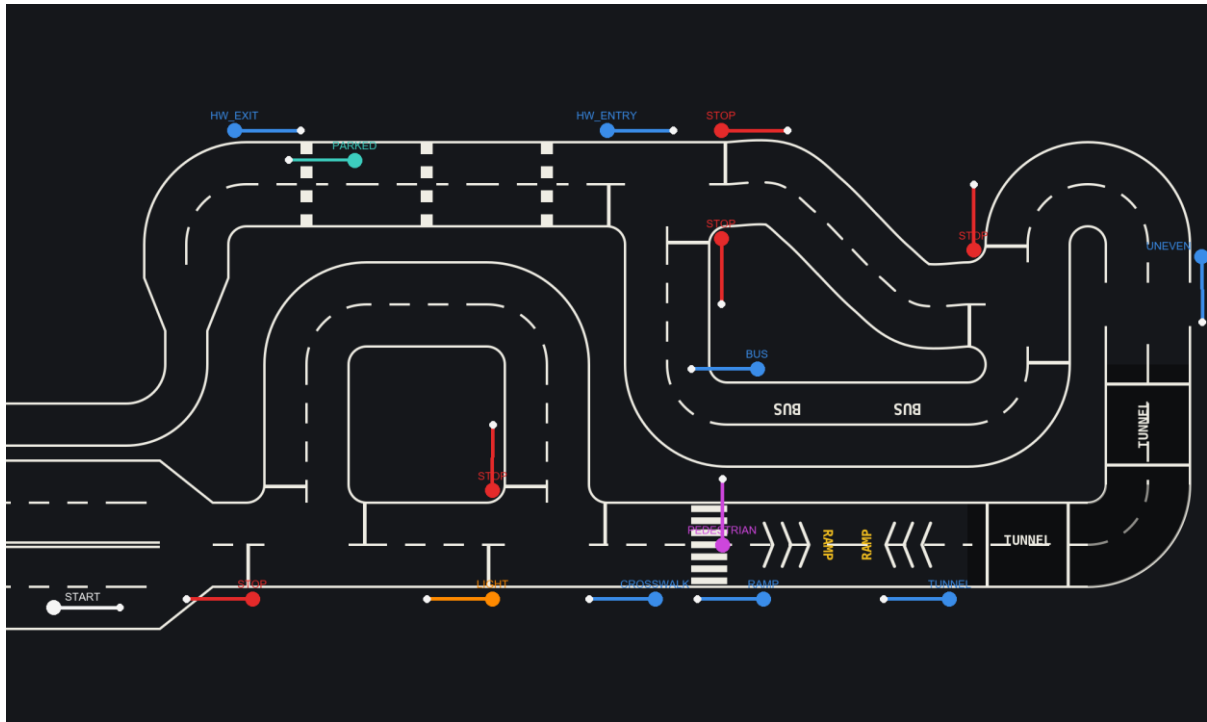


UEH CREATIVE ROBOT CONTEST 2026

Simulation Round

Participant Handbook



The contest track. White is the start pose, orange the traffic light, red the STOP signs, blue the other signs, teal the parked robot, magenta the pedestrian.

At a glance

Contest	UEH Creative Robot Contest 2026
Round	Round 1 — Simulation (qualifying)
Format	Individual. You submit alone, under your own name.
Robot	TurtleBot3 Waffle — differential drive, camera + LiDAR
Track	about 9 × 4 m of road on a 10 × 6 m floor; one traffic light, twelve signs, a parked robot, a pedestrian, ramp, tunnel
Platform	ROS 2 Humble + Gazebo Classic 11, inside Docker
Language	English throughout: code comments, report, video narration

Contents

Part 1	Simulation Pack	What is in the pack, how to install and run it, the topics your code talks to, and what to do when something breaks.
Part 2	Assignment	The task itself, what is allowed and what is not, how runs are scored, and how AI assistance is handled.
Part 3	Submission	The deadline, repository layout, the report, the video, the marks, and the checklist to run before you submit.

About this document. This is the complete documentation for the simulation round: how to install and run the simulator, what you have to build, and what to hand in. The same text also ships inside the simulation pack, and that copy is the authoritative one — if the two ever disagree, the pack wins.

Commands are shown in boxes like this one and are typed exactly as written. Unless a section says otherwise, they run from the root of the pack folder on your own machine.

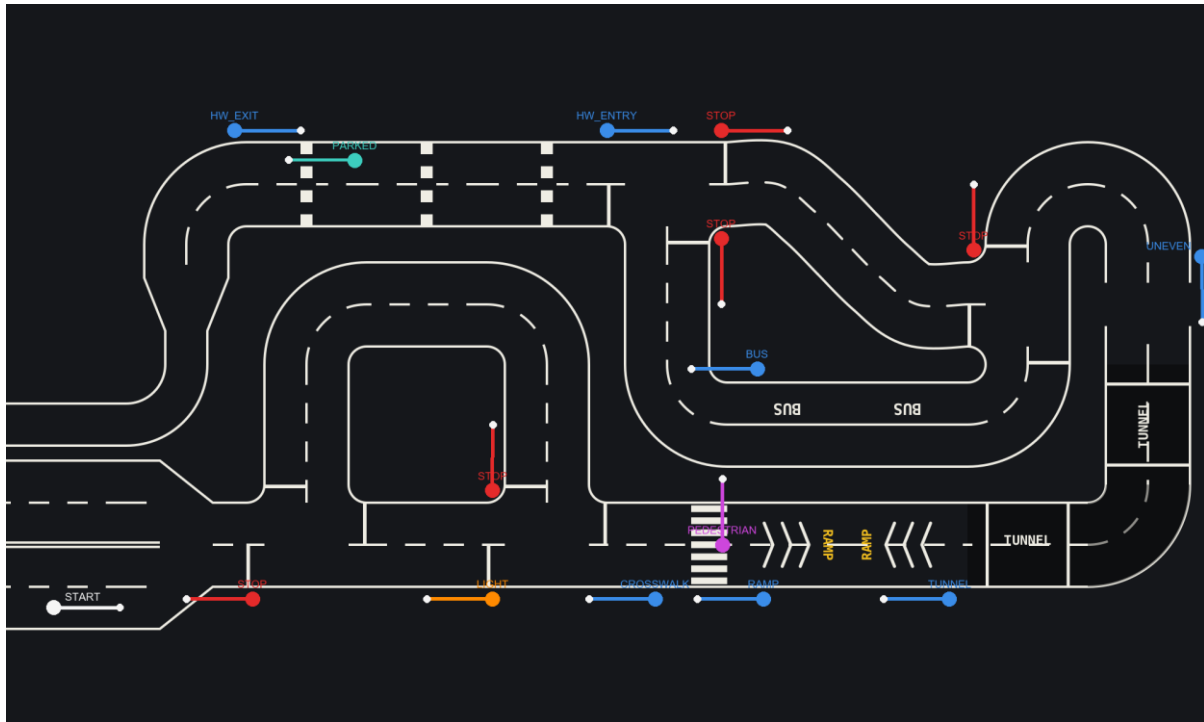
PART 1

Simulation Pack

Installing the simulator, running it, and finding your way around what it publishes.

Round 1, individual assignment. This pack contains the simulator, the track and a starter node. See **Part 2, Assignment** for the task and **Part 3, Submission** for what to hand in.

Built on ROS 2 Humble and Gazebo Classic 11. The robot is a TurtleBot3 Waffle (differential drive), painted orange so it is easy to spot. The circuit is about 9×4 m of road, laid out on a 10.00×6.00 m floor.



Track layout

White is the start pose, orange the traffic light, red the STOP signs, blue the other signs, teal the parked robot and magenta the pedestrian. The short line shows which way each one faces — for the pedestrian, the way it crosses.

1. What you need

Linux with Docker. 8 GB of RAM and 12 GB of free disk. No dedicated GPU.

Your user has to be in the `docker` group, otherwise the very first command fails with a permission error on the Docker socket:

```
sudo usermod -aG docker $USER
```

Log out and back in for that to take effect.

Your distribution does not matter — everything runs inside the container, which is always Ubuntu 22.04 with ROS 2 Humble and Gazebo 11. Tested on Ubuntu 22.04 and 24.04.

No desktop needed either: `up-headless` starts its own virtual display, so a bare server or an SSH session works. A desktop is only for watching the Gazebo window.

On Windows use WSL2 with Ubuntu. macOS will not work.

2. Getting it running

```
bash scripts/run_docker.sh build
bash scripts/run_docker.sh compile
bash scripts/run_docker.sh up
```

build pulls a large image and takes about 10 minutes the first time. After **up**, Gazebo appears on your desktop.

Other subcommands: **up-headless** (no window, no host display), **sh** (shell in the container), **demo** (drives the robot, saves frames to `~/crc_shots`), **down**. With no desktop, use **up-headless** and view the run through **ros2 run crc_sim snapshot**, which writes camera frames to PNG.

If you already have ROS 2 Humble installed and would rather not use Docker:

```
mkdir -p ~/crc_sim_ws && cp -r src scripts ~/crc_sim_ws/ && bash
~/crc_sim_ws/scripts/setup_linux.sh
```

3. Checking it works

ROS runs inside the container, not on your host machine. Open a second terminal and get a shell in there first:

```
bash scripts/run_docker.sh sh
```

Every **ros2** command below is typed inside that shell.

```
ros2 run crc_sim sensor_check
```

You should see four topics with a rate next to each:

```
/scan          10.0    360 rays, valid=16, front=inf m
/camera/image_raw 28.9    640x480 rgb8
/odom          29.1    x=+0.00 y=+0.00 v=+0.00 w=+0.00
/imu           97.5    az=+9.81 m/s^2
```

NO DATA YET on any line means that sensor is not publishing; see section 7.

To get a feel for the track, drive it yourself:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Watching the camera live

Three ways, all from a container shell. The first is usually what you want:

```
ros2 run rqt_image_view rqt_image_view /camera/image_raw
ros2 launch crc_sim sim.launch.py rviz:=true
```

RViz also shows the LiDAR scan and the robot model beside the image.

With no display there is no live view; save frames and open them afterwards:

```
ros2 run crc_sim snapshot --ros-args -p out_dir:=/tmp/shots -p period:=1.0 -p count:=20
```

4. Topics

Topic	Type	Notes
<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	in, drives the robot
<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	RGB 640×480 at 30 Hz

Topic	Type	Notes
/camera/camera_info	sensor_msgs/CameraInfo	
/scan	sensor_msgs/LaserScan	360 rays, 0.12 to 3.5 m
/odom	nav_msgs/Odometry	wheel odometry, drifts
/imu	sensor_msgs/Imu	
/sky_cam/image_raw	sensor_msgs/Image	top-down view, ground truth
/traffic_lights	std_msgs/String	lamp colours, ground truth
/traffic_light/<id>/state	std_msgs/String	one lamp, ground truth
/automobile/semaphores	std_msgs/String	lamp colours, ground truth
/model_states	gazebo_msgs/ModelStates	exact pose of every model, ground truth
/link_states	gazebo_msgs/LinkStates	exact pose of every link, ground truth
/joint_states	sensor_msgs/JointState	wheel angles
/tf, /tf_static	tf2_msgs/TFMessage	odom → base_footprint

The six ground-truth feeds are there so you can check your perception against the truth while developing. Your solution must not subscribe to any of them during the graded run (see **Part 2, Assignment §3**).

/odom is not ground truth: it is integrated from wheel rotation, so it starts at zero wherever the robot happens to be and drifts as you drive.

The world origin is the centre of the board, +X to the right and +Y up as you see it on the track image, with the surface at $z = 0$. The robot starts on the START line in the bottom-left corner, at world $(-4.60, -2.02)$, pointing along +X. Note that /odom reports a different frame: it reads $(0, 0)$ at the moment the simulation starts, not the world position.

5. Writing your code

Open `src/crc_sim/crc_sim/starter_node.py` and fill in `control()`. It runs 20 times a second and hands you:

```
self.image      # BGR image, 480x640x3, or None
self.scan      # raw LaserScan
self.range_at(deg) # distance in one direction: 0 ahead, 90 left, -90 right
self.x, self.y, self.yaw
self.drive(v, w)  # v m/s forward, w rad/s counter-clockwise
self.stop()
```

Run it from a container shell:

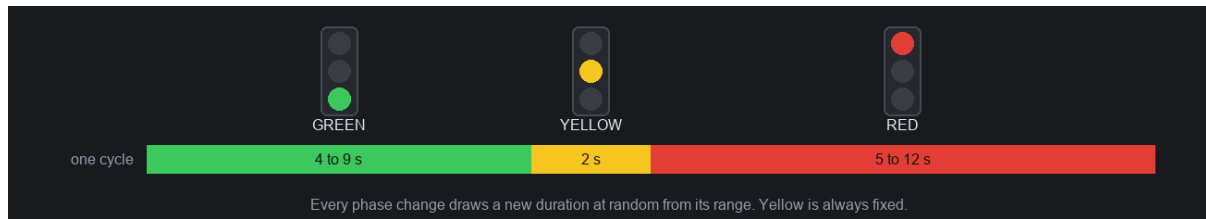
```
ros2 run crc_sim starter
```

`src/` is mounted into the container and built with `--symlink-install`, so editing a `.py` file only means restarting the node — no rebuild.

Add your own nodes and dependencies, or replace `starter_node.py` entirely. The one hard requirement is that a single command starts your solution.

6. What is on the track

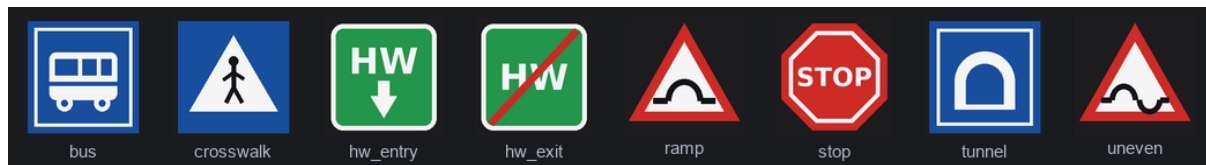
One traffic light, 350 mm tall, on a junction approach. Random schedule, redrawn at every phase change: green 4–9 s, yellow always 2 s, red 5–12 s. You cannot predict it, so you have to see it.



Traffic light cycle

Twelve signs, 70 × 70 mm plates on 200 mm poles: five STOP, one pedestrian crossing, one bus stop at the bus bay, one hump warning at the ramp, one tunnel sign before the tunnel mouth, one uneven-road warning before a stretch where the centre line is missing, and the pair that marks where the highway section begins and ends.

Eight plate types, all below and all on this track, in the international shapes: red triangle warns, blue panel informs. The graded track uses the same eight in a different mix.

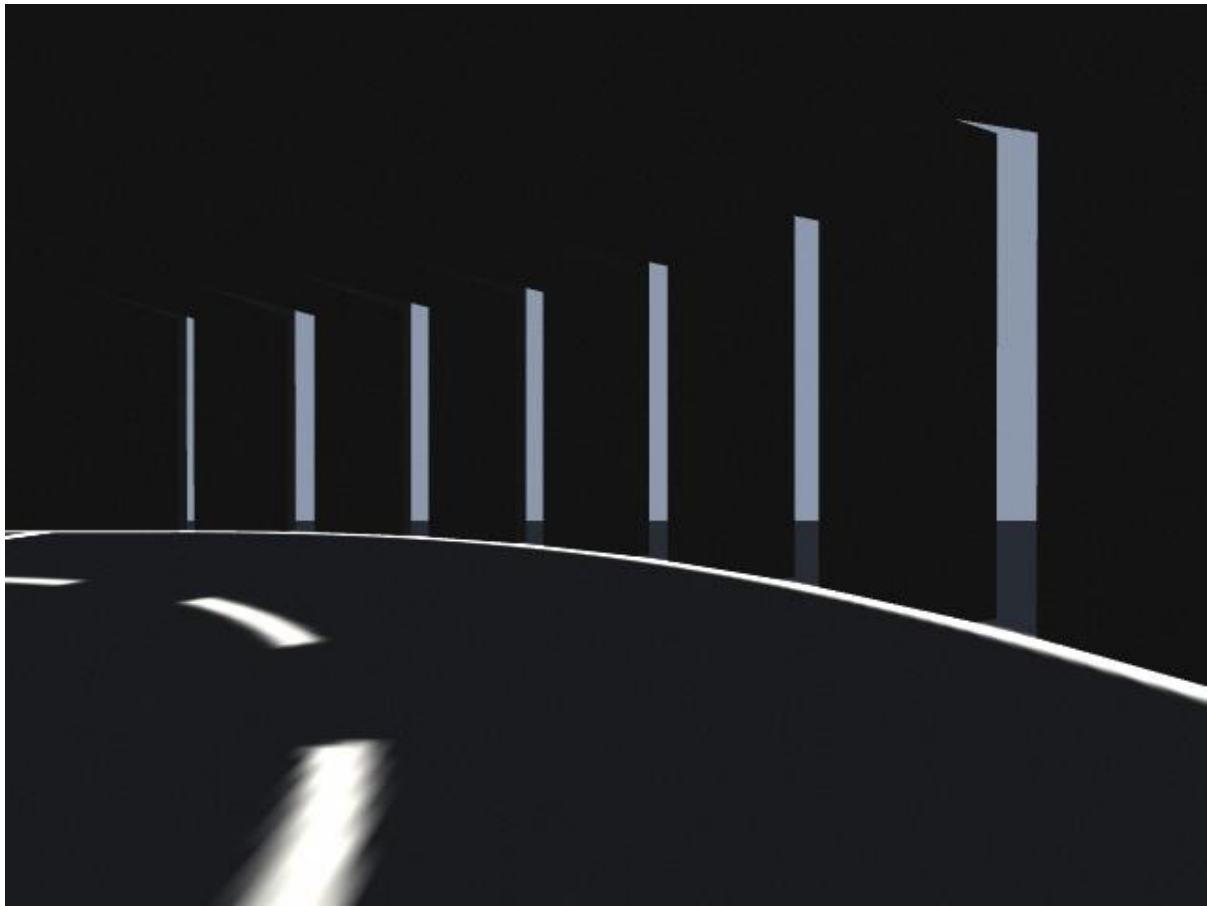


Traffic signs

A blue robot stands still in your lane on the highway section between the two HW signs. The road there is 715 mm wide in two lanes with a dashed centre line, and there is no oncoming traffic, so overtaking means crossing into the other lane — about 48 mm of clearance on each side while you are alongside it.

A pedestrian, 200 mm tall, crosses the zebra: 15 s at one kerb, about 6 s to cross, 15 s at the other, repeating. Camera and LiDAR both see it.

Also a 2.0 m ramp rising 50 mm, and a dark tunnel over a straight and the bend after it.



Dark tunnel

The road inside is genuinely dimmed, and light leaks between the wall segments on the curve, so the image flickers between bright and dark as you pass through.

7. When things go wrong

Symptom	What to do
<code>/camera/image_raw</code> never publishes	Gazebo Classic needs a display to render cameras even with no window. Use <code>up-headless</code> , which starts Xvfb in the container. If you launch by hand on a desktop, keep <code>DISPLAY</code> set and mount <code>/tmp/.X11-unix</code> .
Robot appears as wheels with no body	<code>turtlebot3_description</code> missing. Launch with <code>use_mesh:=false</code> .
Track is flat grey with no markings	Gazebo cannot find the texture. Check <code>\$GAZEBO_MODEL_PATH</code> includes <code>.../crc_sim/models</code> and rebuild.
<code>self.image</code> stays <code>None</code>	<code>apt install ros-humble-cv-bridge python3-opencv</code>
Gazebo is very slow or black	Falling back to software rendering. Try <code>LIBGL_ALWAYS_SOFTWARE=1</code> , or cut the load with <code>props:=false skycam:=false</code> .
Gazebo survives Ctrl+C	<code>pkill -9 gzserver gzclient</code>
<code>compile</code> fails with <code>can't copy '...': doesn't exist</code>	The build cache still lists a file you deleted or renamed. Wipe it and build again: <code>bash scripts/run_docker.sh down && docker volume rm crc_build && bash scripts/run_docker.sh compile</code>

Symptom	What to do
A file in your folder that you cannot delete	The container runs as root, so anything it creates in a mounted folder belongs to root. Remove it with <code>docker run --rm -v "\$HOME":/h alpine rm -rf /h/<path></code> .

8. Launch arguments

`run_docker.sh up` uses the defaults. To change them, stop it, open a container shell and launch by hand:

```
ros2 launch crc_sim sim.launch.py <name>:=<value>
```

Argument	Default	What it does
<code>gui</code>	<code>true</code>	open the Gazebo window
<code>rviz</code>	<code>false</code>	open RViz2
<code>track_scale</code>	<code>1.0</code>	1.0 is the real board, 10 × 6 m
<code>camera_pitch</code>	<code>0.0</code>	camera tilt down, in radians
<code>lidar_range</code>	<code>3.5</code>	LiDAR range in metres
<code>lights, signs, props, skycam</code>	<code>true</code>	spawn each group, independently
<code>use_mesh</code>	<code>true</code>	<code>false</code> draws the robot as plain boxes
<code>tunnel_seg</code>	<code>0.13</code>	wall segment length on the tunnel bend; <code>0.20</code> seals the light leaks
<code>body_color</code>	<code>Gazebo/Orange</code>	robot body colour
<code>ped_dwell</code>	<code>15.0</code>	seconds the pedestrian waits at each kerb
<code>ped_trigger</code>	<code>0.0</code>	metres; above zero the pedestrian also steps out when you get that close
<code>x, y, yaw</code>	<code>auto, auto, 0.0</code>	start pose

Note the tolerances: lanes are 350 mm wide and the Waffle is 306 mm across the wheels, leaving 22 mm each side. That is the hardest part of the task. You can give yourself room while developing with `track_scale:=2.0`, but graded runs use `1.0`.

PART 2

Assignment

What you have to build, the rules the graded run is held to, and how the marks are worked out.

UEH Creative Robot Contest 2026, Round 1. Individual work: you submit alone, under your own name.

1. The task

Drive a TurtleBot3 Waffle from START to FINISH using only the sensors on the robot. In one continuous run, handle:

1. **Lane keeping.** Lanes are 350 mm wide. On one stretch the centre line is missing; a warning sign marks it.
2. **Traffic lights.** Read the lamp colour off the camera. Stop on red, and on yellow unless you are already inside the junction.
3. **Traffic signs.** React to the ones you meet. STOP means a full stop of at least 2 seconds before the junction.
4. **Pedestrian crossing.** A pedestrian crosses the zebra. Stop while they are on the road.
5. **Ramp.** Cross the 2.0 m ramp without losing the lane.
6. **Dark tunnel.** Keep going where the road is dimmed and the light flickers.
7. **Overtaking.** A second robot stands still in your lane on the highway section. Go around it and pull back in.

You are not expected to finish all seven. Marks are partial.

2. What you get, and what you do not

You get the simulator, the track, a starter node with the ROS plumbing done, and ground-truth topics to check your perception against while you develop.

The track used for grading is not this one. Which signs appear and where, the light position and its timing, where the parked robot stands, and the start pose will all differ. The road layout stays the same. Hard-coded coordinates will score close to nothing.

3. Rules

Allowed: any library from apt or pip (OpenCV, NumPy, PyTorch, ONNX Runtime, YOLO, anything else); training your own models, including on data you generate from this simulation; AI coding assistants (see §5). Discuss ideas with other participants, but do not share or copy code.

Not allowed during the graded run:

- subscribing to `/traffic_lights`, `/traffic_light/*`, `/automobile/semaphores`, `/sky_cam/*`, `/model_states` or `/link_states`
- calling `/get_entity_state`, `/set_entity_state`, `/spawn_entity`, `/delete_entity` or any other Gazebo introspection service
- reading the world, model or config files at run time to recover object positions
- changing anything under `src/crc_sim/worlds/`, `src/crc_sim/models/` or `src/crc_sim/config/`

This is checked automatically. Using any of it scores zero for the whole performance component, with no appeal.

`/odom` is fine — it is a real sensor. It drifts, and the graded start pose differs, so odometry alone will not get you round.

4. Scoring

The **performance component** below is marked out of 100 and is worth 50% of the round. The other 50% is the report and the video; see **Part 3, Submission §5**.

Your code is run three times on the graded track with a different random seed each time. Your performance score is the median.

Component	Points
Distance completed along the route	30
Lane keeping, RMS error against the centreline	20
Traffic lights obeyed	15
Traffic signs detected and obeyed	15
Ramp, tunnel and crossing handled	10
Obstacles avoided	10

Overtaking means crossing the dashed centre line. That is allowed, and lane error is not counted while you are alongside the parked robot — but return to your lane afterwards, and leaving the road still costs you.

Penalties: leaving the road –5 each, collision –10 each, running a red light –15 each. A run still going after 5 minutes is stopped and scored on what it achieved.

A run that fails to start, crashes, or does not launch with the single command you gave us scores zero. Test that command before you submit.

5. AI assistants

Allowed, and you do not need to hide it. What we assess is whether you understand what you handed in.

Your report states what was AI-generated, what you wrote, and what you changed and why. In the video you explain your own code and change a parameter live, so a statement that does not match what you can explain will show.

6. Getting through to the next round

The Organising Committee selects who goes on to the hardware round from the submitted results, and announces it afterwards.

That round is held in person, on campus at UEH in Ho Chi Minh City. This round is open to students from any university; the next one is not remote. Teams are formed there from the qualifiers and may mix people from different schools. If getting to campus would be a problem, tell us before you put the time in. Dates come with the results.

Your code carries over — the hardware robot exposes the same topics.

7. Advice

Do lane keeping first and nothing else. Take the bottom third of the camera image, threshold it, find the centroid of the white pixels, steer on the error from the image centre. With distance completed, that is half the performance component.

Do not find the traffic light by colour alone. The five STOP plates are the same saturated red as the red lamp, both highway plates the same saturated green, and at 200–270 mm they sit at almost exactly the lamp's height. What separates the light is that it changes: watch a candidate for a second and see whether its colour switches.

Signs are harder. Most on this track are STOP, which is tempting to special-case, but the graded track carries a different mix.

Go into the tunnel early. A threshold that only works in the bright sections is the classic way to lose those marks.

Do not tune against one start position. Move the robot with the `x`, `y` and `yaw` launch arguments and check that your code recovers.

You are working alone. Two capabilities done properly beat seven done badly.

PART 3

Submission

What to hand in, in what shape, by when — and the checks to run before you do.

Individual submission, under your own name. Everything in English: code comments, report, video narration.

1. Deadline

15 September 2026, 23:59 (UTC+7).

Late work is not accepted. Submit early; you can re-submit if you improve it.

2. What to hand in

One private Git repository (GitHub or GitLab) with the judges added as collaborators. Name it after your school and yourself, not a student number:

```
<school>_<your-full-name>/
├── README.md           how to run your solution, in one command
├── REPORT.pdf          20 pages maximum
├── VIDEO.md            link to the video
├── src/                your code
├── models/             trained weights, if you have any
└── analysis/           the scripts that produced the figures in your report
```

Plain ASCII, no spaces or accents: `UEH_Nguyen-Van-An`, `HCMUT_Tran-Thi-Binh`. Repeat your name, school and faculty in the first lines of your `README.md`.

Your `README.md` must give exactly one command that starts your solution against an already-running simulation, for example:

```
ros2 launch my_solution run.launch.py
```

We start the simulator, then run that command. If it fails on a clean machine you score zero for performance, so test it somewhere that has never seen your project. List extra dependencies as `apt` or `pip` commands.

Commit as you go. A single commit the night before is not disqualifying, but it gives us nothing to go on.

3. Report (20 pages max, PDF)

Suggested split, leaving two pages for title, contents and references:

1. Approach, with one architecture diagram — 2 pages
2. Perception: lanes, lights, signs, with real images from your runs — 5
3. Control: how perception becomes `/cmd_vel` — 3
4. Results, with plots — 5
5. What does not work, and why — 2
6. AI usage statement — 1

Twenty is a ceiling, not a target. Padding is obvious and costs marks.

Every number needs a figure behind it, produced by your own script from your own recorded run, with that script in `analysis/`. A claim with no evidence scores nothing.

Section 5 is marked seriously. A report claiming everything works loses marks.

4. Video (6 minutes, one take)

Screen recording with your voice, in English, with your face visible in a corner. One continuous take: no cuts, no edits, no speeding up.

Time	On screen
0:00–0:30	Your name, your school and your faculty
0:30–2:00	Your algorithm, scrolling through your own code
2:00–4:00	The simulation running live, at normal speed
4:00–5:00	Change one parameter on camera, re-run, explain what changed
5:00–6:00	Limitations, and what you would do with more time

The 4:00–5:00 segment carries the most weight: it takes a minute if you built the thing. Talk, do not read a script.

5. Marks

Performance on the graded track 50%, report 25%, video and English delivery 25%.

There is no interview. Everything is judged from what you submit, which is why the video matters: it is the only place we see you explain your own code.

English is marked on clarity, not accent. You lose marks for saying something you do not understand, never for pronunciation.

6. Academic honesty

Discuss ideas freely. Code has to be yours, or AI-assisted and declared as such. All submissions are compared against each other; two sharing substantial code are both referred to the Committee, whoever wrote it first.

7. Before you submit

- ☐ The repository is named `<school>_<your-full-name>` and your name, school and faculty are in its `README.md`.
- ☐ Cloned the repo onto a clean machine and ran the one command. It worked.
- ☐ Nothing subscribes to any of the ground-truth topics banned in ASSIGNMENT §3.
- ☐ No Gazebo entity-state or spawn service calls.
- ☐ Nothing under `src/crc_sim/worlds/`, `src/crc_sim/models/` or `src/crc_sim/config/` was modified.
- ☐ Nothing reads those files at run time to recover object positions either.
- ☐ No track coordinates are hard-coded anywhere; the graded track is a different one.
- ☐ Tested from a different start pose (`x`, `y`, `yaw`).
- ☐ Tested at `track_scale:=1.0`, not only scaled up.
- ☐ Every figure in the report has its script in `analysis/`.
- ☐ The video is one take and includes the live parameter change.

- ☐ The AI usage statement is accurate.